

Phase 5 Completion Report – Apex Programming (Developer)

1. Classes & Objects

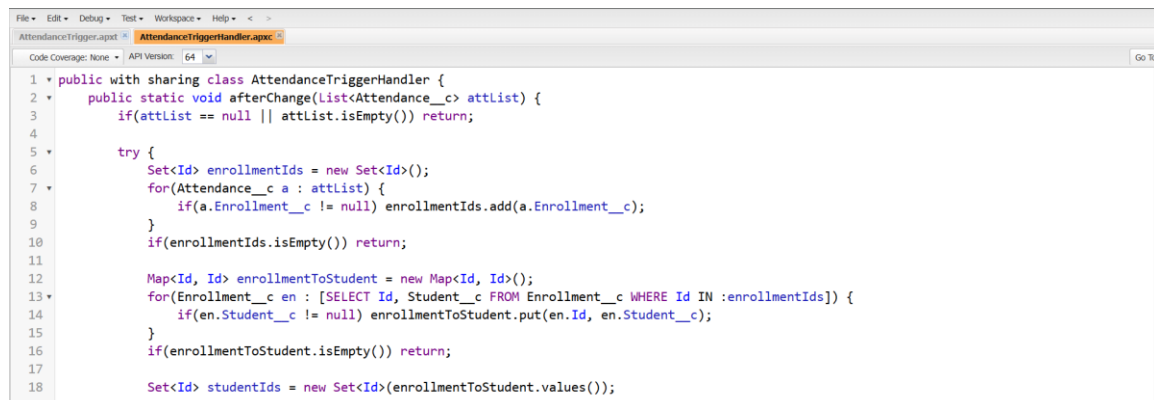
Apex classes were created to handle business logic and calculations for the student management system. Each class encapsulates specific functionality.

Steps to Create Apex Class:

1. Developer Console → File → New → Apex Class.
2. Enter Class Name and click OK.
3. Write Apex code implementing the required logic.
4. Save the class.

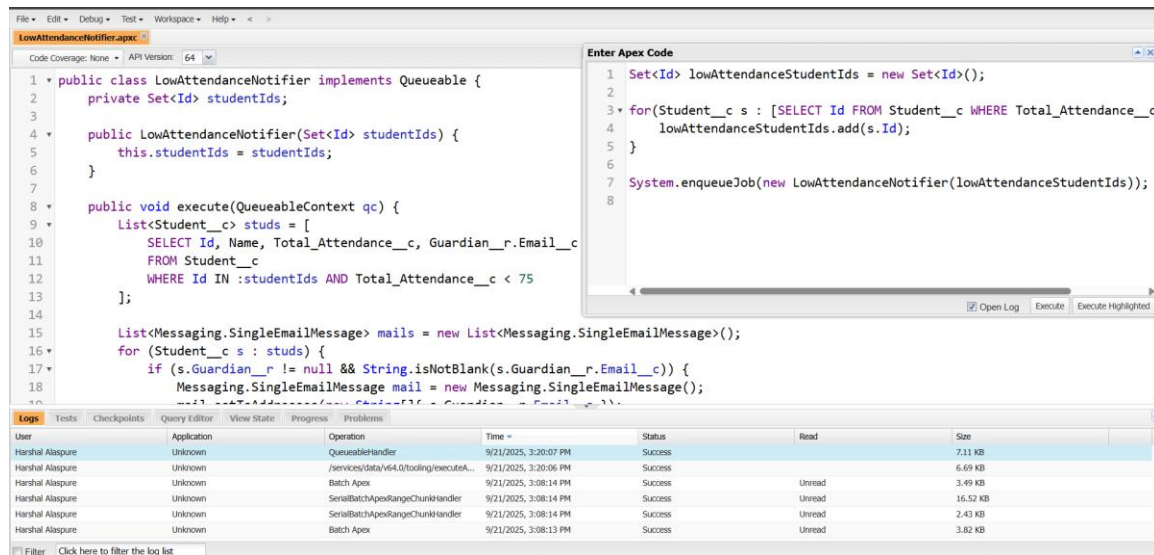
Classes Created:

- AttendanceTriggerHandler: Calculates student attendance percentage and updates the Student__c object.



```
1 public with sharing class AttendanceTriggerHandler {
2     public static void afterChange(List<Attendance__c> attList) {
3         if(attList == null || attList.isEmpty()) return;
4
5         try {
6             Set<Id> enrollmentIds = new Set<Id>();
7             for(Attendance__c a : attList) {
8                 if(a.Enrollment__c != null) enrollmentIds.add(a.Enrollment__c);
9             }
10            if(enrollmentIds.isEmpty()) return;
11
12            Map<Id, Id> enrollmentToStudent = new Map<Id, Id>();
13            for(Enrollment__c en : [SELECT Id, Student__c FROM Enrollment__c WHERE Id IN :enrollmentIds]) {
14                if(en.Student__c != null) enrollmentToStudent.put(en.Id, en.Student__c);
15            }
16            if(enrollmentToStudent.isEmpty()) return;
17
18            Set<Id> studentIds = new Set<Id>(enrollmentToStudent.values());
19        }
```

- LowAttendanceNotifier: Sends email alerts to guardians if attendance falls below threshold.



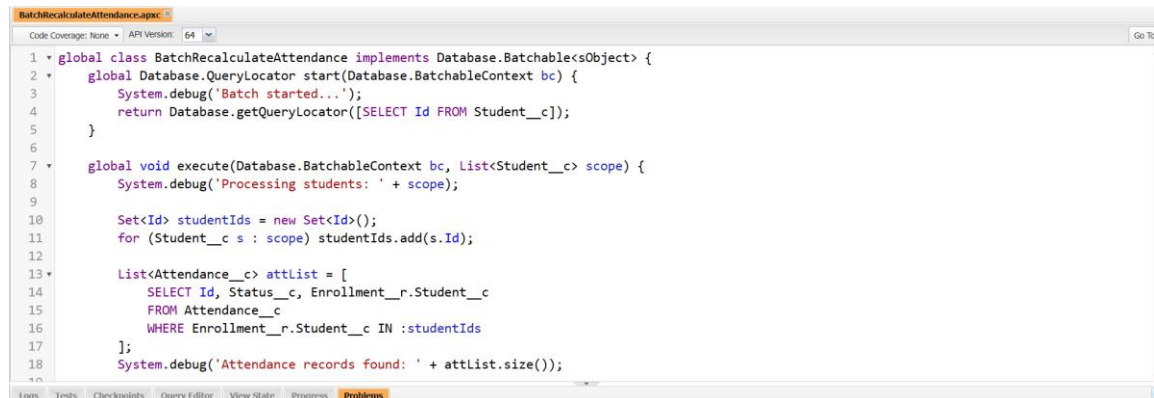
The screenshot displays the Salesforce IDE with the `LowAttendanceNotifier.apex` class open. The class implements the `Queueable` interface and contains the following code:

```
1 public class LowAttendanceNotifier implements Queueable {
2     private Set<Id> studentIds;
3
4     public LowAttendanceNotifier(Set<Id> studentIds) {
5         this.studentIds = studentIds;
6     }
7
8     public void execute(QueueableContext qc) {
9         List<Student__c> studs = [
10             SELECT Id, Name, Total_Attendance__c, Guardian__r.Email__c
11             FROM Student__c
12             WHERE Id IN :studentIds AND Total_Attendance__c < 75
13         ];
14
15         List<Messaging.SingleEmailMessage> mails = new List<Messaging.SingleEmailMessage>();
16         for (Student__c s : studs) {
17             if (s.Guardian__r != null && String.isNotBlank(s.Guardian__r.Email__c)) {
18                 Messaging.SingleEmailMessage mail = new Messaging.SingleEmailMessage();
19                 mail.setToAddresses(new String[] { s.Guardian__r.Email__c });
20                 mail.setSubject('Low Attendance Alert for ' + s.Name);
21                 mail.setBody('Your child, ' + s.Name + ' has a low attendance percentage of ' + s.Total_Attendance__c + '%. Please ensure they attend school regularly to avoid negative impacts on their education.');
22                 mails.add(mail);
23             }
24         }
25         Messaging.sendEmail(mails);
26     }
27 }
```

Below the code editor, the 'Logs' tab is selected, showing a table of execution logs. The table has columns for User, Application, Operation, Time, Status, Read, and Size. The logs show successful execution of the `LowAttendanceNotifier` class for multiple students.

User	Application	Operation	Time	Status	Read	Size
Harshal Alaspure	Unknown	QueueableHandler	9/21/2025, 3:20:07 PM	Success		7.11 KB
Harshal Alaspure	Unknown	/services/data/v64.0/tooling/execute...	9/21/2025, 3:20:06 PM	Success		6.69 KB
Harshal Alaspure	Unknown	Batch Apex	9/21/2025, 3:20:14 PM	Success	Unread	3.49 KB
Harshal Alaspure	Unknown	SerialBatchApexRangeChunkHandler	9/21/2025, 3:08:14 PM	Success	Unread	16.52 KB
Harshal Alaspure	Unknown	SerialBatchApexRangeChunkHandler	9/21/2025, 3:08:14 PM	Success	Unread	2.43 KB
Harshal Alaspure	Unknown	Batch Apex	9/21/2025, 3:08:13 PM	Success	Unread	3.82 KB

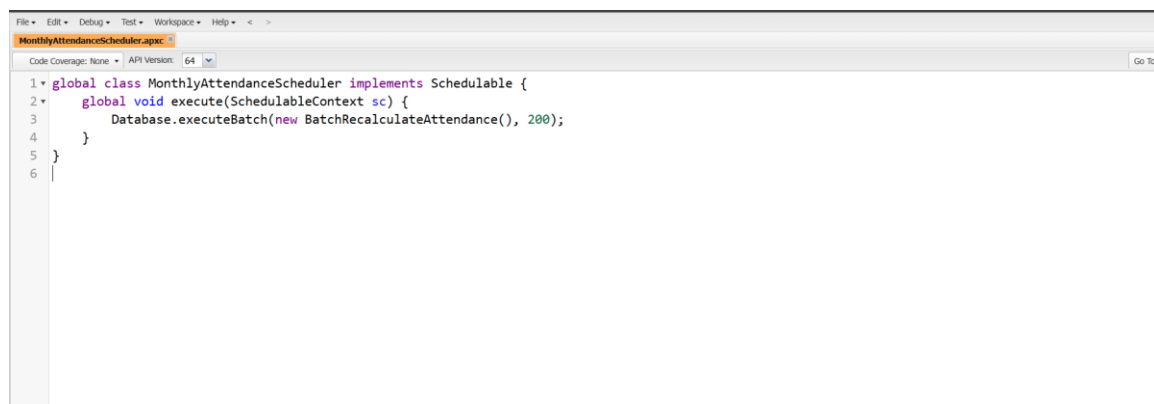
- BatchRecalculateAttendance: Recalculates attendance percentages for all students in bulk.



The screenshot displays the Salesforce IDE with the `BatchRecalculateAttendance.apex` class open. The class implements the `Database.Batchable<Object>` interface and contains the following code:

```
1 global class BatchRecalculateAttendance implements Database.Batchable<Object> {
2     global Database.QueryLocator start(Database.BatchableContext bc) {
3         System.debug('Batch started...');
4         return Database.getQueryLocator([SELECT Id FROM Student__c]);
5     }
6
7     global void execute(Database.BatchableContext bc, List<Student__c> scope) {
8         System.debug('Processing students: ' + scope);
9
10        Set<Id> studentIds = new Set<Id>();
11        for (Student__c s : scope) studentIds.add(s.Id);
12
13        List<Attendance__c> attList = [
14            SELECT Id, Status__c, Enrollment__r.Student__c
15            FROM Attendance__c
16            WHERE Enrollment__r.Student__c IN :studentIds
17        ];
18        System.debug('Attendance records found: ' + attList.size());
19    }
20 }
```

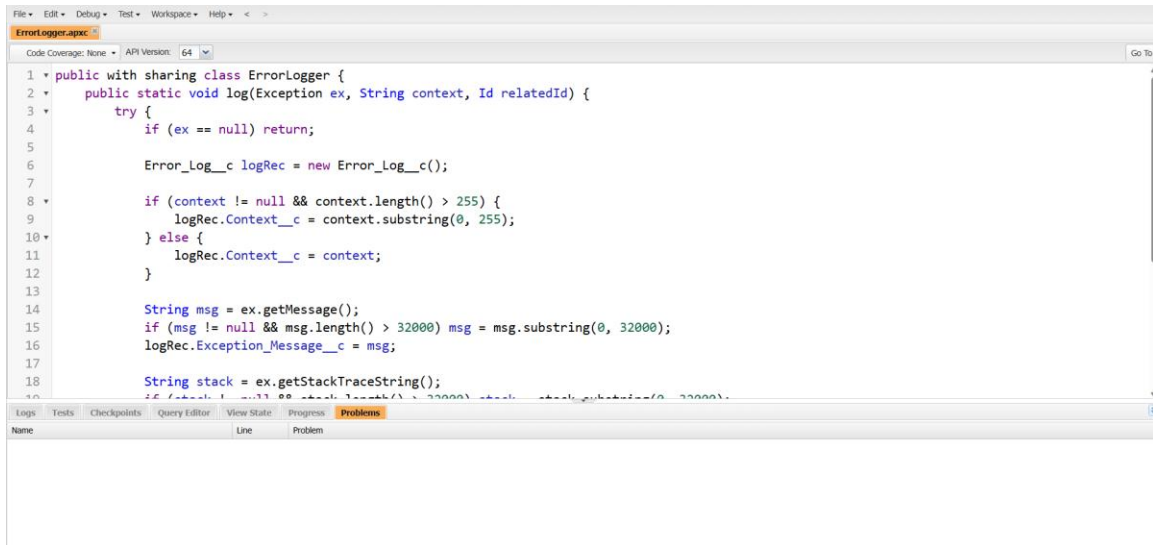
- MonthlyAttendanceScheduler: Schedules monthly batch execution to recalculate attendance.



The screenshot displays the Salesforce IDE with the `MonthlyAttendanceScheduler.apex` class open. The class implements the `Schedulable` interface and contains the following code:

```
1 global class MonthlyAttendanceScheduler implements Schedulable {
2     global void execute(SchedulableContext sc) {
3         Database.executeBatch(new BatchRecalculateAttendance(), 200);
4     }
5 }
6
```

- - ErrorLogger – Logs exceptions to Error_Log__c for debugging and auditing.

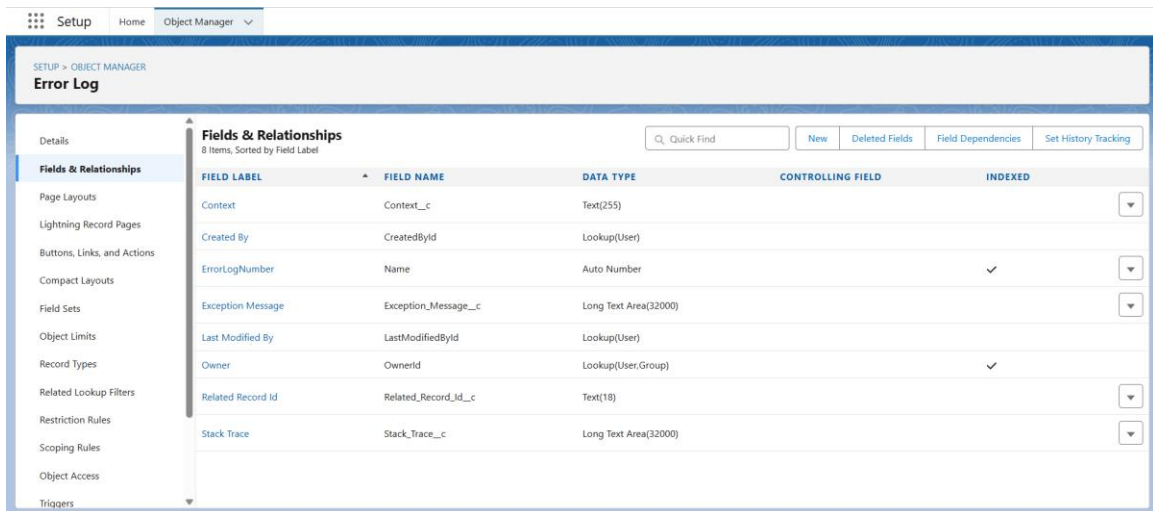


```

1 public with sharing class ErrorLogger {
2     public static void log(Exception ex, String context, Id relatedId) {
3         try {
4             if (ex == null) return;
5
6             Error_Log__c logRec = new Error_Log__c();
7
8             if (context != null && context.length() > 255) {
9                 logRec.Context__c = context.substring(0, 255);
10            } else {
11                logRec.Context__c = context;
12            }
13
14            String msg = ex.getMessage();
15            if (msg != null && msg.length() > 32000) msg = msg.substring(0, 32000);
16            logRec.Exception_Message__c = msg;
17
18            String stack = ex.getStackTraceString();
19            if (stack != null && stack.length() > 32000) stack = stack.substring(0, 32000);
20            logRec.Stack_Trace__c = stack;
21        } catch (Exception e) {
22            // Log the exception
23        }
24    }
25 }

```

- - Error Log Object was created same as mentioned steps in Phase 3 for this Phase



FIELD LABEL	FIELD NAME	DATA TYPE	CONTROLLING FIELD	INDEXED
Context	Context__c	Text(255)		
Created By	CreatedById	Lookup(User)		
ErrorLogNumber	Name	Auto Number		✓
Exception Message	Exception_Message__c	Long Text Area(32000)		
Last Modified By	LastModifiedById	Lookup(User)		
Owner	OwnerId	Lookup(User.Group)		✓
Related Record Id	Related_Record_Id__c	Text(18)		
Stack Trace	Stack_Trace__c	Long Text Area(32000)		

2. Apex Triggers

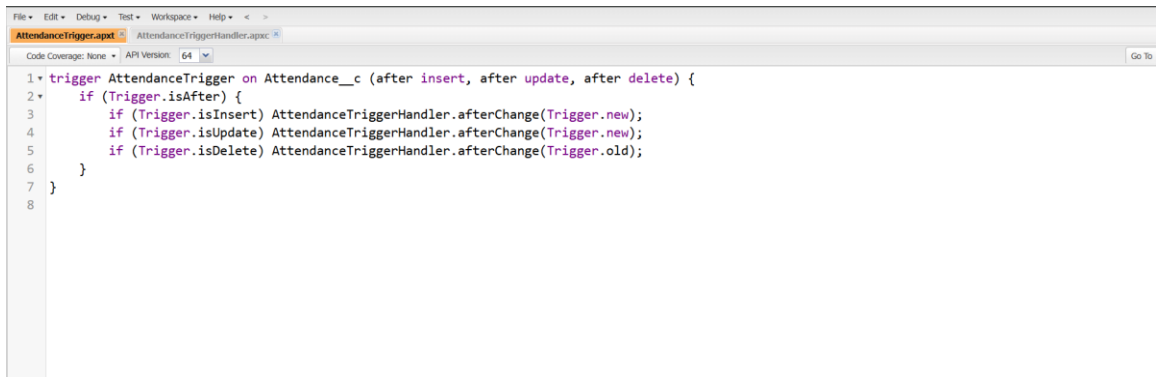
-AttendanceTrigger

Object : Attendance__c

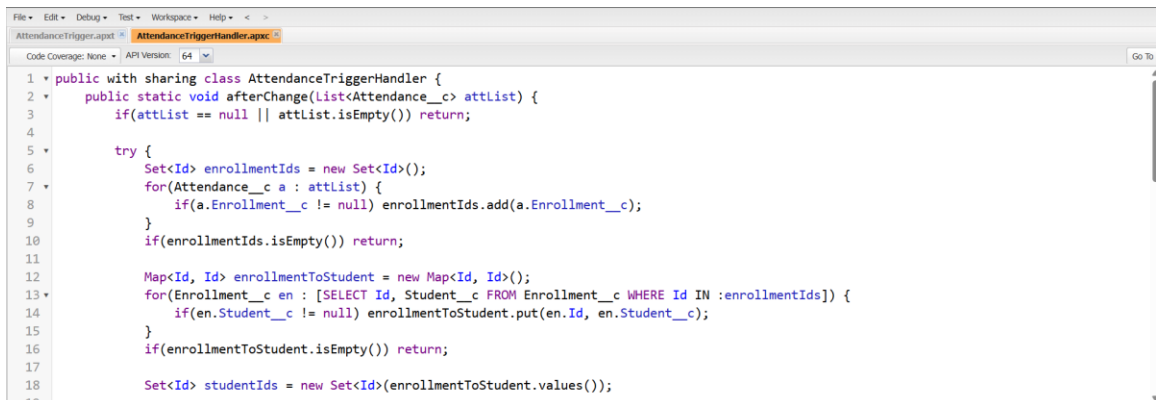
Trigger was created to automatically update attendance percentages when Attendance__c records are inserted, updated, or deleted.

Trigger: AttendanceTrigger on Attendance__c (after insert, update, delete)

- Calls AttendanceTriggerHandler.afterChange() to update attendance percentages.



```
1 trigger AttendanceTrigger on Attendance__c (after insert, after update, after delete) {
2     if (Trigger.isAfter) {
3         if (Trigger.isInsert) AttendanceTriggerHandler.afterChange(Trigger.new);
4         if (Trigger.isUpdate) AttendanceTriggerHandler.afterChange(Trigger.new);
5         if (Trigger.isDelete) AttendanceTriggerHandler.afterChange(Trigger.old);
6     }
7 }
8
```



```
1 public with sharing class AttendanceTriggerHandler {
2     public static void afterChange(List<Attendance__c> attList) {
3         if(attList == null || attList.isEmpty()) return;
4
5         try {
6             Set<Id> enrollmentIds = new Set<Id>();
7             for(Attendance__c a : attList) {
8                 if(a.Enrollment__c != null) enrollmentIds.add(a.Enrollment__c);
9             }
10            if(enrollmentIds.isEmpty()) return;
11
12            Map<Id, Id> enrollmentToStudent = new Map<Id, Id>();
13            for(Enrollment__c en : [SELECT Id, Student__c FROM Enrollment__c WHERE Id IN :enrollmentIds]) {
14                if(en.Student__c != null) enrollmentToStudent.put(en.Id, en.Student__c);
15            }
16            if(enrollmentToStudent.isEmpty()) return;
17
18            Set<Id> studentIds = new Set<Id>(enrollmentToStudent.values());
19        }
20    }
21 }
```

Steps to Create Trigger:

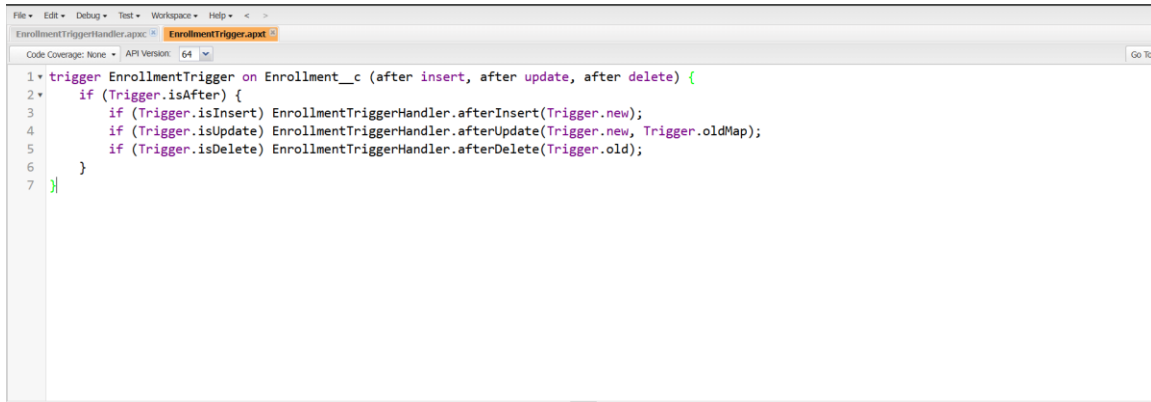
1. Developer Console → File → New → Apex Trigger.
2. Select Object: Attendance__c.
3. Write Trigger code calling the handler.
4. Save the trigger.

-EnrollmentTrigger

Object: Enrollment__c

Purpose: Handle automation related to enrollments, e.g., updating related records or sending notifications when a student is enrolled, updated, or removed from a course.

Handler Called: EnrollmentTriggerHandler (afterInsert, afterUpdate, afterDelete methods)



```
1 trigger EnrollmentTrigger on Enrollment__c (after insert, after update, after delete) {  
2     if (Trigger.isAfter) {  
3         if (Trigger.isInsert) EnrollmentTriggerHandler.afterInsert(Trigger.new);  
4         if (Trigger.isUpdate) EnrollmentTriggerHandler.afterUpdate(Trigger.new, Trigger.oldMap);  
5         if (Trigger.isDelete) EnrollmentTriggerHandler.afterDelete(Trigger.old);  
6     }  
7 }
```

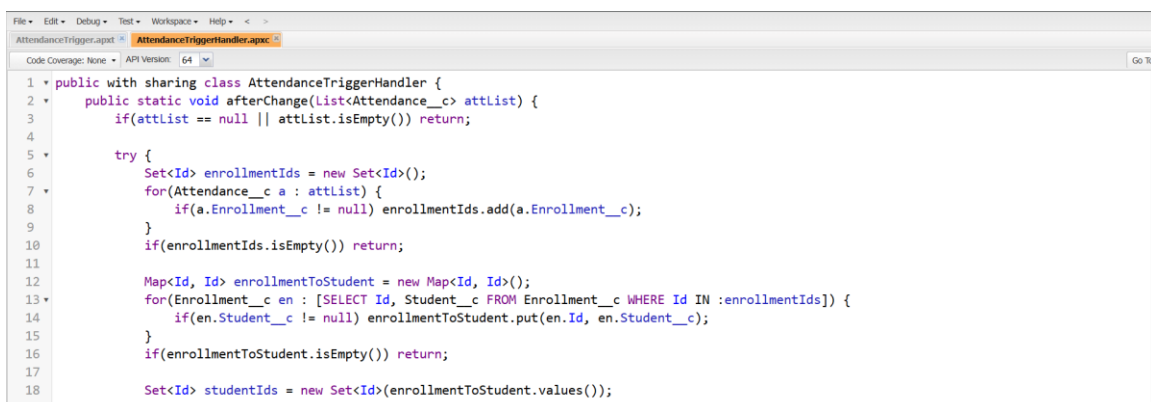
Steps to Create Trigger:

1. Developer Console → File → New → Apex Trigger.
2. Select Object: Enrollment__c.
3. Write trigger code calling the handler:

3. Trigger Design Pattern

- Trigger handler classes (e.g. AttendanceTriggerHandler) were implemented to keep trigger logic separate from the trigger definition.
- This ensures reusability and easier maintenance.

AttendanceTriggerHandler Screenshot



```
1 public with sharing class AttendanceTriggerHandler {  
2     public static void afterChange(List<Attendance__c> attList) {  
3         if(attList == null || attList.isEmpty()) return;  
4  
5         try {  
6             Set<Id> enrollmentIds = new Set<Id>();  
7             for(Attendance__c a : attList) {  
8                 if(a.Enrollment__c != null) enrollmentIds.add(a.Enrollment__c);  
9             }  
10            if(enrollmentIds.isEmpty()) return;  
11  
12            Map<Id, Id> enrollmentToStudent = new Map<Id, Id>();  
13            for(Enrollment__c en : [SELECT Id, Student__c FROM Enrollment__c WHERE Id IN :enrollmentIds]) {  
14                if(en.Student__c != null) enrollmentToStudent.put(en.Id, en.Student__c);  
15            }  
16            if(enrollmentToStudent.isEmpty()) return;  
17  
18            Set<Id> studentIds = new Set<Id>(enrollmentToStudent.values());  
19        }  
20    }
```

EnrollmentTriggerHandler Screenshot

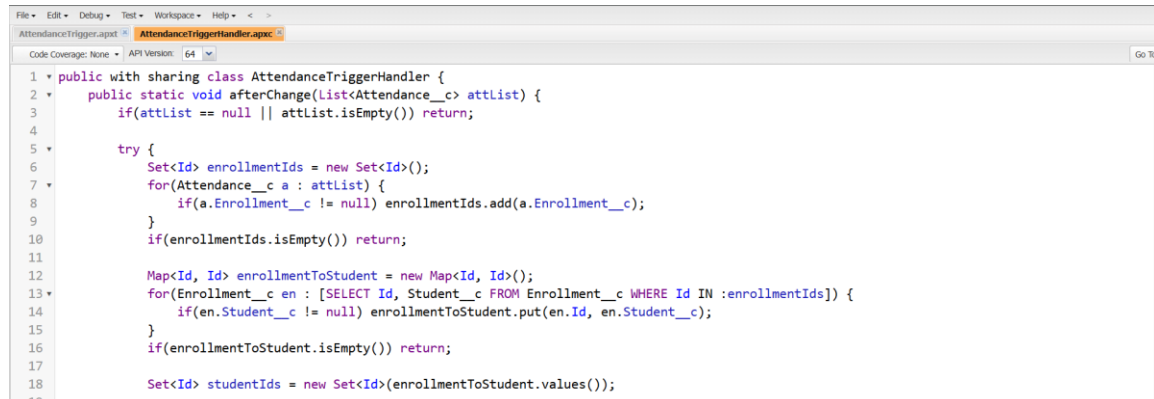


```
1 public with sharing class EnrollmentTriggerHandler {
2
3     public static void afterInsert(List<Enrollment__c> newList) {
4         updateStudentCourseCounts(newList);
5     }
6
7     public static void afterUpdate(List<Enrollment__c> newList, Map<Id, Enrollment__c> oldMap) {
8         updateStudentCourseCounts(newList);
9     }
10
11    public static void afterDelete(List<Enrollment__c> oldList) {
12        updateStudentCourseCounts(oldList);
13    }
14
15    private static void updateStudentCourseCounts(List<Enrollment__c> enrolls) {
16        try {
17            Set<Id> studentIds = new Set<Id>();
18            for (Enrollment__c e : enrolls) {
19                if (e.Student__c != null) studentIds.add(e.Student__c);
20            }
21        } catch (Exception ex) {
22            // Handle exception
23        }
24    }
25 }
```

The screenshot shows an IDE window titled "EnrollmentTriggerHandler.apex" with a code editor displaying the Apex class. The class is named "EnrollmentTriggerHandler" and is marked as "with sharing". It contains four static methods: "afterInsert", "afterUpdate", "afterDelete", and "updateStudentCourseCounts". The "afterInsert", "afterUpdate", and "afterDelete" methods all call "updateStudentCourseCounts". The "updateStudentCourseCounts" method is private and takes a list of "Enrollment__c" objects. It uses a try-catch block to handle exceptions, creating a set of student IDs and iterating over the enrollments to add them to the set. The IDE interface includes a menu bar (File, Edit, Debug, Test, Workspace, Help), a toolbar (Code Coverage, API Version, Go To), and a bottom panel with tabs for Logs, Tests, Checkpoints, Query Editor, View State, Progress, and Problems.

4. SOQL & SOSL

- SOQL used in AttendanceTriggerHandler and BatchRecalculateAttendance to fetch Enrollment and Attendance records.
- AggregateResult used to calculate totals and present counts.



```
1 public with sharing class AttendanceTriggerHandler {
2     public static void afterChange(List<Attendance__c> attList) {
3         if(attList == null || attList.isEmpty()) return;
4
5         try {
6             Set<Id> enrollmentIds = new Set<Id>();
7             for(Attendance__c a : attList) {
8                 if(a.Enrollment__c != null) enrollmentIds.add(a.Enrollment__c);
9             }
10            if(enrollmentIds.isEmpty()) return;
11
12            Map<Id, Id> enrollmentToStudent = new Map<Id, Id>();
13            for(Enrollment__c en : [SELECT Id, Student__c FROM Enrollment__c WHERE Id IN :enrollmentIds]) {
14                if(en.Student__c != null) enrollmentToStudent.put(en.Id, en.Student__c);
15            }
16            if(enrollmentToStudent.isEmpty()) return;
17
18            Set<Id> studentIds = new Set<Id>(enrollmentToStudent.values());
```



```
1 global class BatchRecalculateAttendance implements Database.Batchable<Object> {
2     global Database.QueryLocator start(Database.BatchableContext bc) {
3         System.debug('Batch started...');
4         return Database.getQueryLocator([SELECT Id FROM Student__c]);
5     }
6
7     global void execute(Database.BatchableContext bc, List<Student__c> scope) {
8         System.debug('Processing students: ' + scope);
9
10        Set<Id> studentIds = new Set<Id>();
11        for (Student__c s : scope) studentIds.add(s.Id);
12
13        List<Attendance__c> attList = [
14            SELECT Id, Status__c, Enrollment__r.Student__c
15            FROM Attendance__c
16            WHERE Enrollment__r.Student__c IN :studentIds
17        ];
18        System.debug('Attendance records found: ' + attList.size());
```

5. Collections: List, Set, Map

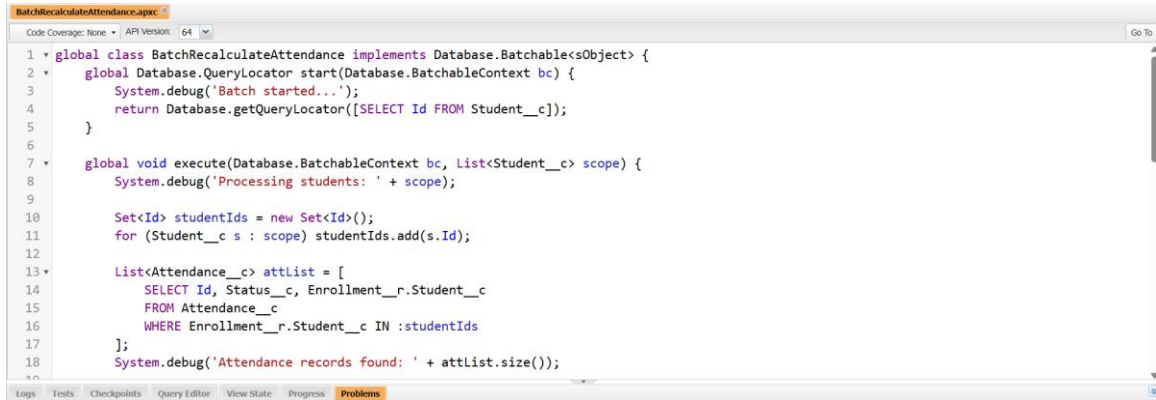
- List used to batch update students.
- Set used to store unique Enrollment and Student IDs.
- Map<Id, Id> used to map Enrollment to Student.
- Map<Id, Decimal> used to store attendance percentages.

6. Control Statements

- if, for loops, try-catch used throughout handler and batch classes for flow control and error handling.

7. Batch Apex

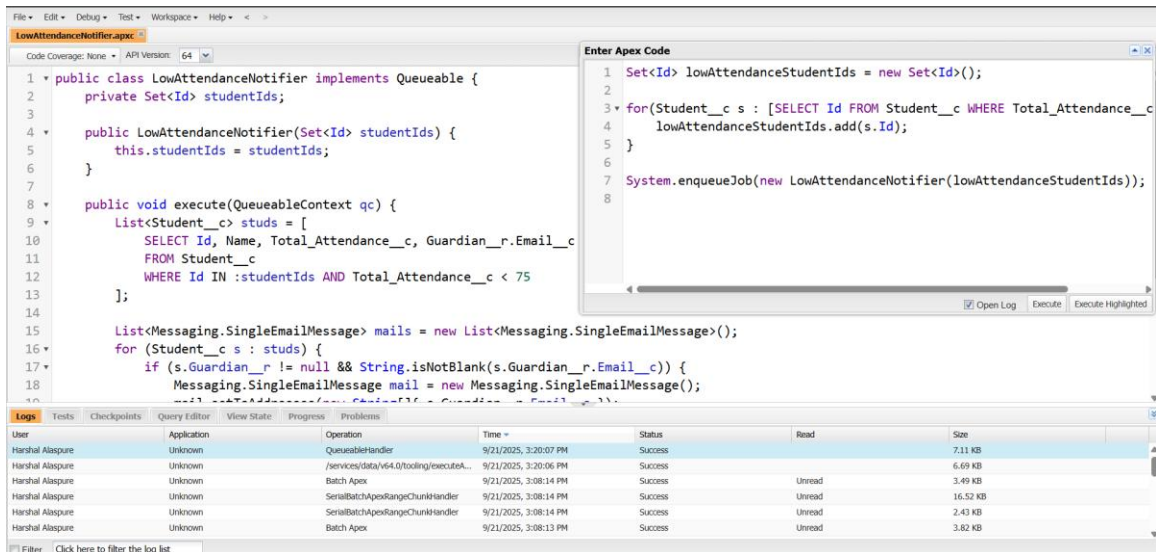
- BatchRecalculateAttendance class implements Database.Batchable to recalculate attendance for all students.
- Developer Console → Execute Anonymous to run batch manually.



```
1 global class BatchRecalculateAttendance implements Database.Batchable<Object> {
2     global Database.QueryLocator start(Database.BatchableContext bc) {
3         System.debug('Batch started...');
4         return Database.getQueryLocator([SELECT Id FROM Student__c]);
5     }
6
7     global void execute(Database.BatchableContext bc, List<Student__c> scope) {
8         System.debug('Processing students: ' + scope);
9
10        Set<Id> studentIds = new Set<Id>();
11        for (Student__c s : scope) studentIds.add(s.Id);
12
13        List<Attendance__c> attlist = [
14            SELECT Id, Status__c, Enrollment__r.Student__c
15            FROM Attendance__c
16            WHERE Enrollment__r.Student__c IN :studentIds
17        ];
18        System.debug('Attendance records found: ' + attlist.size());
19    }
20 }
```

8. Queueable Apex

- LowAttendanceNotifier implements Queueable to send asynchronous email notifications to guardians for low attendance.
- Accepts a Set of Student IDs and processes in background.



```
1 public class LowAttendanceNotifier implements Queueable {
2     private Set<Id> studentIds;
3
4     public LowAttendanceNotifier(Set<Id> studentIds) {
5         this.studentIds = studentIds;
6     }
7
8     public void execute(QueueableContext qc) {
9         List<Student__c> studs = [
10             SELECT Id, Name, Total_Attendance__c, Guardian__r.Email__c
11             FROM Student__c
12             WHERE Id IN :studentIds AND Total_Attendance__c < 75
13         ];
14
15         List<Messaging.SingleEmailMessage> mails = new List<Messaging.SingleEmailMessage>();
16         for (Student__c s : studs) {
17             if (s.Guardian__r != null && String.isNotBlank(s.Guardian__r.Email__c)) {
18                 Messaging.SingleEmailMessage mail = new Messaging.SingleEmailMessage();
19                 mail.setToAddresses([s.Guardian__r.Email__c]);
20                 mail.setSubject('Low Attendance Notification');
21                 mail.setBody('Your child ' + s.Name + ' has a low attendance of ' + s.Total_Attendance__c);
22                 mails.add(mail);
23             }
24         }
25         Messaging.sendEmail(mails);
26     }
27 }
```

User	Application	Operation	Time	Status	Read	Size
Harshal Alaspure	Unknown	QueueableHandler	9/21/2025, 3:20:07 PM	Success		7.11 KB
Harshal Alaspure	Unknown	/services/data/v44.0/feeding/executeA...	9/21/2025, 3:20:06 PM	Success		6.69 KB
Harshal Alaspure	Unknown	Batch Apex	9/21/2025, 3:08:14 PM	Success	Unread	3.49 KB
Harshal Alaspure	Unknown	SerialBatchApexRangeChunkHandler	9/21/2025, 3:08:14 PM	Success	Unread	16.52 KB
Harshal Alaspure	Unknown	SerialBatchApexRangeChunkHandler	9/21/2025, 3:08:14 PM	Success	Unread	2.43 KB
Harshal Alaspure	Unknown	Batch Apex	9/21/2025, 3:08:13 PM	Success	Unread	3.82 KB

9. Scheduled Apex

- MonthlyAttendanceScheduler schedules BatchRecalculateAttendance to run monthly.

```
File • Edit • Debug • Test • Workspace • Help • < • >
MonthlyAttendanceScheduler.apex
Code Coverage: None | API Version: 64
Go To

1 global class MonthlyAttendanceScheduler implements Schedulable {
2     global void execute(SchedulableContext sc) {
3         Database.executeBatch(new BatchRecalculateAttendance(), 200);
4     }
5 }
6
```

- Setup → Apex Classes → Schedule Apex or via Developer Console.

The screenshot shows the Salesforce Setup interface. On the left, the 'Setup' menu is open, and 'Apex Classes' is selected under 'Custom Code'. The main content area is titled 'Schedule Apex' and contains the following configuration:

- Job Name:** Monthly Attendance Recalc
- Apex Class:** MonthlyAttendanceScheduler
- Schedule Using:** Schedule Builder (selected), Cron Expression
- Frequency:** Monthly (selected), Weekly
- Start:** 9/21/2025 (9/21/2025)
- End:** 10/21/2025 (9/21/2025)
- Preferred Start Time:** 11:00 AM

Buttons for 'Save' and 'Cancel' are visible at the top and bottom of the configuration area.

10. Future Methods

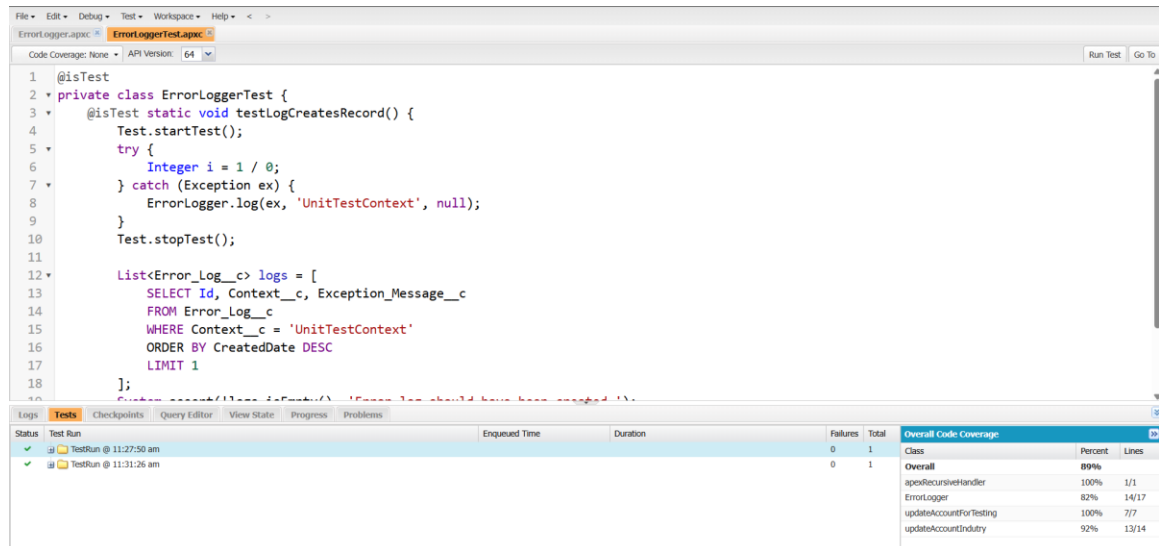
- Not implemented in this phase as Queueable Apex handled asynchronous processing.

11. Exception Handling

- try-catch blocks implemented in AttendanceTriggerHandler and Queueable classes.
- System.debug used to log exceptions for troubleshooting.

12. Test Classes

- ErrorLoggerTestClass: Tests the ErrorLogger Class and trigger working.
- Ensures code coverage > 75%.



13. Asynchronous Processing

- Batch Apex, Queueable Apex, and Scheduled Apex implemented to handle bulk data and background processing.

14. Process Flow Summazed

A. AttendanceTriggerHandler

Developer Console → File → New → Apex Class → Write Code → Save → Trigger fires on Attendance insert/update/delete → Handler calculates percentage → Update Student__c.Total_Attendance__c.

B. LowAttendanceNotifier

Developer Console → File → New → Apex Class → Write Queueable code → Save → Execute Queueable passing Set of low-attendance students → Emails sent to Guardian__c.Email__c.

C. BatchRecalculateAttendance

Developer Console → File → New → Apex Class → Write Batchable code → Save → Execute via Developer Console or Scheduled Apex → Attendance percentages recalculated for all students.

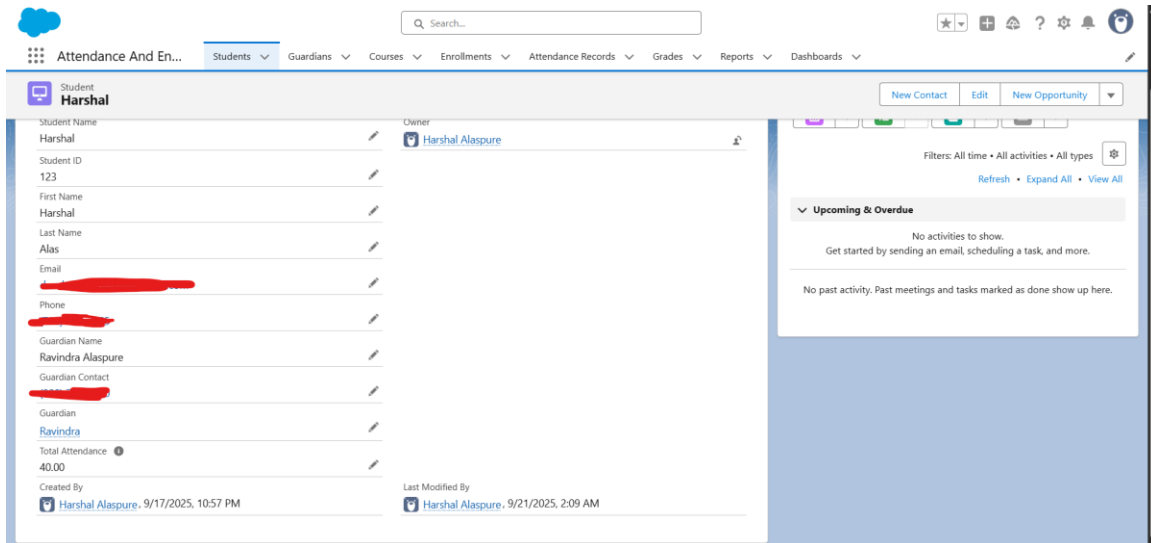
D. MonthlyAttendanceScheduler

Developer Console → File → New → Apex Class → Write Schedulable code → Save → Setup → Apex Classes → Schedule Apex → Runs monthly → Executes BatchRecalculateAttendance.

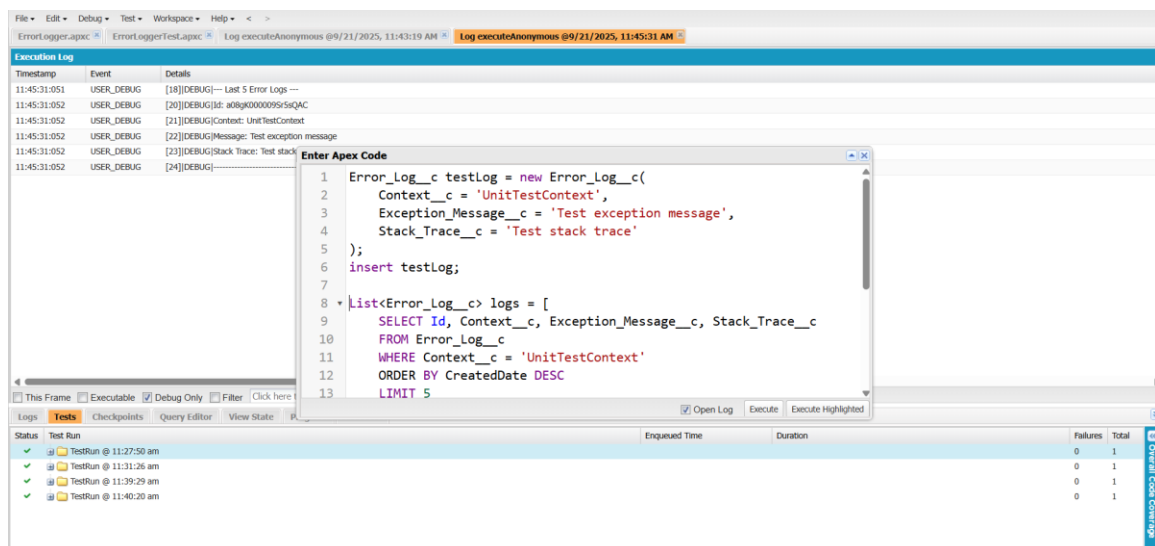
15. Test and Result Screenshots

Here are all the screenshots of the triggering outputs

- When AttendanceTrigger, AttendanceTriggerHandler worked and I noted attendance for a few days, the “Total Attendance” field of Student_c got autofilled via triggering event.



- When ErrorLoggerClass worked and the AnonymousWindow used for taking a test on some error data, here's the whole output screenshot.



- When the queueable task was created and ran, the automated mail to the guardian email of respective student has been sent if their attendance is found below 75%.

